Chapter 14

Back-of-the-Envelope Estimation

Phase 1 · Fundamentals · Estimated time: 1.5 hours

1. Concept From First Principles

Before you can design a system correctly, you need a rough sense of its scale — is this 100 requests per second or 100,000? Is this 1GB of data or 1PB? The answer completely changes the right architecture (a single Postgres instance is fine for one scale and hopeless for another).
Back-of-the-envelope estimation is the skill of quickly, roughly calculating these numbers from a few given or assumed facts — using simple arithmetic, not precision.

> **Backend engineer analogy**
>
> This is the same instinct you use when deciding whether a nightly cron job report needs to process "a few thousand rows" (just loop in PHP, fine) versus "50 million rows" (needs chunking, queues, maybe a different approach entirely). HLD interviews just ask you to do this sizing exercise out loud, explicitly, before you design anything.

It's worth being honest about why this skill feels uncomfortable at first: most backend work rewards precision — a query is correct or it isn't, a test passes or fails. Estimation deliberately asks you to be comfortable being *roughly* right, fast, in front of someone watching. That discomfort is normal and fades quickly with practice; the exercises at the end of this chapter exist specifically to build that muscle before it's tested under interview pressure.

The core method

1. **Clarify the metric you actually need** — requests per second? Storage per day? Bandwidth?
2. **Break it into a simple formula** — e.g., storage/day = new records/day × average size per record.
3. **Plug in round numbers** — use 10, 100, 1000, 1 million; never chase false precision.
4. **Do the arithmetic step by step, out loud** — the interviewer is watching your reasoning process, not just the final number.
5. **Sanity-check and use the result** — state what the number implies for your design (e.g., "200K QPS means one database definitely won't cut it").

Useful constants worth memorizing

- Seconds in a day ≈ 86,400 — round to 100,000 for quick mental math.
- Seconds in a year ≈ 31.5 million — round to 30 million.
- 1 million requests/day ≈ ~12 requests/second average (but peak traffic is often 2–10× average — always account for peak, not just average).
- 1 KB ≈ a short JSON object; 1 MB ≈ a compressed photo; these rough sizes are usually good enough.

The single most common mistake candidates make: calculating *average* load and stopping there. Real systems must survive *peak* load — a food delivery app's Friday 8pm traffic can be many times its 3am traffic. Always ask or state a reasonable peak-to-average ratio explicitly.

Beyond QPS and storage, a third estimate worth practicing is **bandwidth** — how much data flows in/out per second, which determines network and CDN costs and capacity. The formula mirrors the others: bandwidth = QPS × average response size. A service doing 1,000 requests/second with a 5KB average response needs roughly 5MB/second of outbound bandwidth — a number that matters when deciding whether responses need compression, pagination, or a CDN in front of them.

2. Real-World Example / Case Study

Imagine estimating for a Zomato/Swiggy-scale order system: 10 million orders per day. Average QPS = 10,000,000 / 100,000 seconds ≈ 100 orders/second. But dinner-rush peak traffic might be 8× average, so peak QPS ≈ 800/second — a number that changes whether a single database write path can keep up, and directly motivates decisions like async processing (Chapter 5) and caching (Chapter 8) for anything not strictly needed synchronously.

For storage: if each order record is roughly 1KB and you get 10 million orders/day, that's ~10GB/day, ~3.65TB/year — a number that's very manageable for a single well-indexed relational database, which is exactly the kind of estimation that lets you confidently defend "we don't need to shard yet" in an interview, backed by actual numbers instead of a guess.

A second worked example: estimating cache memory needs for Meesho's product catalog. If the catalog has 50 million products and, following the common 80/20 rule, roughly 20% of products (10 million) account for 80% of traffic, and each cached product entry is about 2KB, caching just that hot 20% needs roughly 20GB of memory — a number that immediately tells you whether a single Redis instance is enough or whether you need a clustered cache, well before writing any code.

3. Diagram

```
Estimating QPS and storage for an order system:

  Given: 10,000,000 orders/day, ~1KB per order record

  Average QPS  = 10,000,000 / 100,000 sec  =~  100 orders/sec
  Peak QPS     = Average x 8 (dinner rush assumption)  =~  800 orders/sec

  Daily storage  = 10,000,000 x 1KB  =  ~10 GB/day
  Yearly storage = 10 GB x 365       =  ~3.65 TB/year

  Conclusion drawn: single DB can likely handle the WRITE rate;
  storage growth is manageable without sharding for now.
```

4. Trade-offs Table

Estimation choice	Why it's acceptable	Risk if ignored
Using round numbers (100K sec/day)	Interviewer cares about reasoning, not precision	Wasting interview time on exact arithmetic
Assuming a peak-to-average ratio	Real traffic is never flat; systems must survive peaks	Under-designing for the moment that actually breaks

Estimation choice	Why it's acceptable	Risk if ignored
Stopping at QPS/storage only	Good starting point	Missing bandwidth or memory needs the design also depends on

5. Common Interview Questions

Q1: "Estimate the QPS and storage needs for a URL shortener with 100 million new URLs per month."

Model approach: Convert to per-second write QPS using round numbers, separately estimate read QPS (reads are typically far higher than writes for a URL shortener — every redirect is a read), estimate storage per URL record, project over a few years, and explicitly state the read:write ratio since it drives caching decisions later in the interview.

Q2: "Why did you assume peak traffic is 5x average instead of just using the average?"

Model approach: Explain that systems fail at peak, not average, and that real-world traffic for consumer apps is rarely uniform across 24 hours — citing a believable, product-specific reason (e.g., "food delivery peaks at lunch and dinner") makes the assumption defensible rather than arbitrary.

Q3: "Estimate the bandwidth needed for a video-streaming feature with 50,000 concurrent viewers at 2Mbps each."

Model approach: Multiply directly — 50,000 × 2Mbps = 100,000 Mbps = 100 Gbps of total outbound bandwidth, then use that number to reason out loud about why this is exactly the kind of load a CDN (Chapter 10) is designed to absorb, since serving 100Gbps from a single origin server is unrealistic.

6. Practice Exercises

1. Estimate average and peak QPS for a ride-hailing app with 5 million daily rides, assuming a 4x peak-to-average ratio.
2. Estimate yearly storage for a chat app where 50 million users send an average of 20 messages/day, each message averaging 200 bytes.
3. Practice saying your estimation process out loud, narrating each step, in under 3 minutes for either exercise above.

7. Curated YouTube Videos

Language	Video & Channel	Why it helps
English	"Back of the Envelope Storage Estimates | System Design Interview"	Focused specifically on storage estimation walkthroughs.
English	"Back-of-the-Envelope Calculations: System Design Estimations for Beginners"	Beginner-paced full walkthrough of the estimation method.
Hindi	"What is Back of Envelope calculation with examples" — Chai aur Code	Hindi walkthrough with worked examples.

Language	Video & Channel	Why it helps
Hindi	"8. Back-Of-The-Envelope Estimation for System Design Interview | Capacity Planning of Facebook"	Applies the method to a large, familiar real-world system.

8. Key Takeaways

- Back-of-the-envelope estimation quickly sizes a system's scale before you design its architecture.
- Method: clarify the metric, break into a formula, use round numbers, compute step by step out loud, sanity-check the result.
- Always distinguish average load from peak load — systems fail at peak.
- Being off by 2x is fine; being off by 1000x signals a fundamental misunderstanding — precision doesn't matter, direction does.
- Read:write ratio, QPS, and storage estimates directly justify later decisions (caching, sharding, indexing).
- This is a spoken skill — practice narrating your math, not just computing it silently.